

TokTickIT

Lab 1 full-stack vertical slice

Stack	React, TypeScript, Vite, Bootstrap, Express, Prisma, PostgreSQL
Workflow	Bun workspaces, feature branches, GitHub Issues, Pull Requests
Evidence	Tests, API contracts, database seed, and human-checked App Demo

Contents

1	Answer Part 1: Git Use with Engineering Workflow	2
2	Answer Part 2: Tests	2
3	Answer Part 3: AI Use and Reflection	2
4	Answer Part 4: App Demo	2

1 Answer Part 1: Git Use with Engineering Workflow

The repository uses four parent Lab 1 issues and native sub-issues. Work is performed on feature branches and targets `lab1-staging`; the stable branch is `main`.

Parent issue	Feature branch	Pull Request
#13 foundation	<code>feature/1-project-foundation</code>	PR #44
#14 health	<code>feature/2-health-check</code>	PR #46
#15 categories	<code>feature/3-category-seed</code>	PR #45
#16 category list	<code>feature/4-category-list</code>	PR #47

- Repository: <https://github.com/ovenmakemeheat/toktickit>
- Project board: <https://github.com/users/ovenmakemeheat/projects/1/views/2>
- The parent issue checklists are the acceptance source; native sub-issues carry implementation, testing, and evidence slices.
- Student: GUNTEE DOUNGMANEE (67070501003), <https://github.com/ovenmakemeheat>; peer reviewer: POLWARIT WATTHANAHEMMARAT (67070501067), <https://github.com/MadMax168>.
- Positive peer-review records for PRs #44-#47 are audited with the author responses.
- Full review details are in `docs/lab-01/reviewer.md`; PR #47 remains open for human workflow completion.
- Unresolved automated threads on PRs #44, #45, and #47 are recorded separately and are not peer-review rejections.

2 Answer Part 2: Tests

The test matrix is kept in `docs/lab-01/tests.md`. The application boundary is tested through the exported Express app and user-observable React behavior.

ID	Tool	Proof	Status
API-01	Supertest	Health returns HTTP 200 and the exact Tok-TickIT API payload.	Passing
API-02	Supertest	Categories return seeded ID/name pairs in ascending ID order.	Passing
UI-01	Vitest	TokTickIT heading and Check System action render.	Passing
UI-02	Vitest	Checking transitions to Online and renders returned categories.	Passing
UI-03	Vitest	API failure clears stale categories and shows the safe error.	Passing

Verification command: `bun run verify`. It covers Biome, Lefthook validation, Prisma validation, type checks, both workspace test suites, and both workspace builds.

3 Answer Part 3: AI Use and Reflection

The AI-use record in `docs/lab-01/ai_use.md` identifies OpenAI Codex with GPT-5, nine representative prompts, the supporting tools, and the human-in-the-loop checks. Prompt constraints progressively narrowed the work to the Lab 1 scope, Bun workflow, native issue structure, human-only GitHub state changes, and verifiable browser and PDF evidence.

The process remained human checked: the loading capture was recaptured after visual inspection exposed an incorrect completed state, every PDF page was rendered and inspected, and GitHub review records were checked so positive peer comments were not mislabeled as formal approvals. Unresolved automated threads are recorded as unresolved.

4 Answer Part 4: App Demo

The system check makes two relative API requests through the Vite proxy. The success state proves that health is online and that the four categories came from PostgreSQL. The failure state proves that the user receives a safe error and stale categories are cleared.

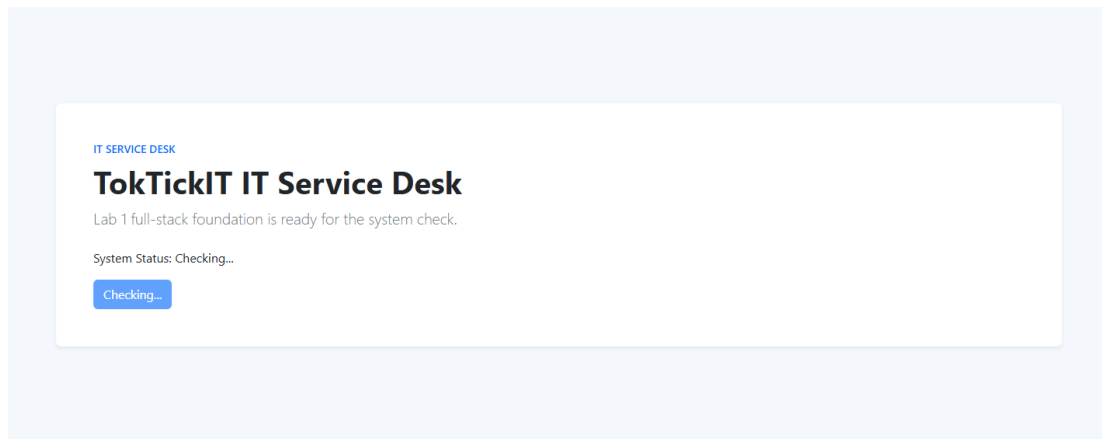


Figure 1: Loading state: the button and status show that the system check is in progress.

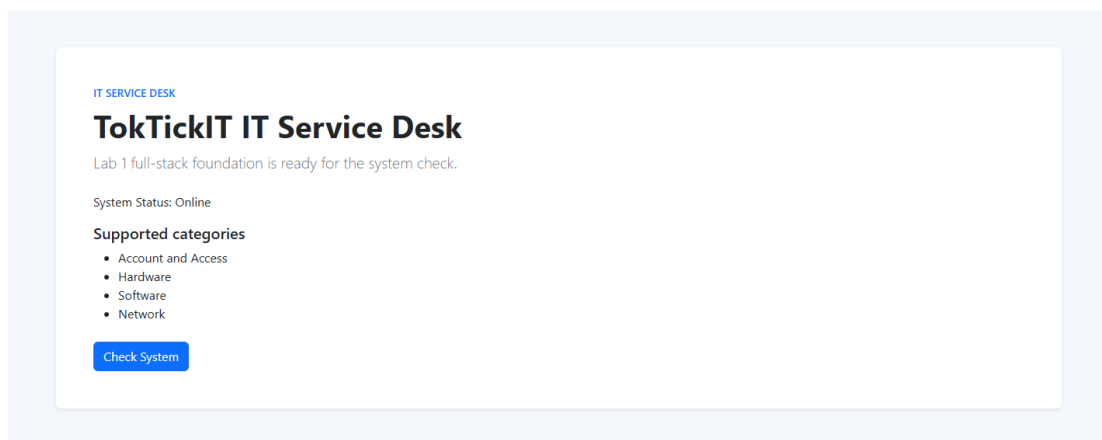


Figure 2: Success state: Online and the database-backed categories are visible in order.

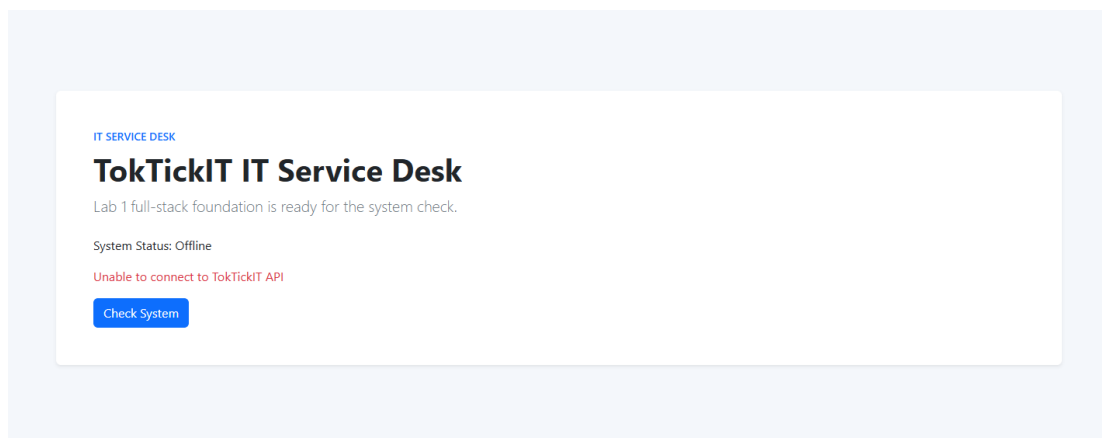


Figure 3: Failure state: the API is unavailable and the user-safe error is shown.

The capture protocol and visual QA checklist are in `docs/lab-01/evidence/capture-checklist.md`. Each final page must be checked for clipped text, unreadable evidence, accidental blank space, and incorrect state labels.